Interability Dependency Graphs

Research method first: focus on regions without sibling-dependent future access.

1. New method: compose ability dependency graphs and identify SQL-compilable regions.

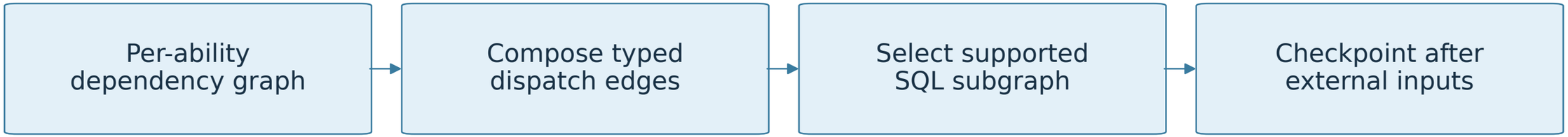
2. Cost model: briefly explain how a prefetch decision estimates benefit and cost.

3. Benchmark set: the full 30-app + 61-operation inventory, followed by the small validation results.

4. Sibling dependency: what changes future access, what can be proved, and what remains outside the first kernel.

1 · From local dependencies to an interability region

Initial analysis target: future accesses do not depend on earlier siblings’ mutable state.



Dependencies include values, predicates, visit targets and effects. Output-only state may stay native when it cannot affect later access.

Implemented kernel: compile-time composition of known typed dispatch; runtime binds origin/actor and validates graph results. General runtime graph composition and arbitrary latest-dependency placement remain research work.

A concrete typed A→B→C traversal

Actual primary fixture; report/count statements retained in native execution.

```
can a with A entry {
    self.visited += 1;
    report here.value;
    visit [->:AB:->[?:B]];
}
can b with B entry {
    self.visited += 1;
    report here.value;
    if here.active {
        visit [->:BC:->[?:C]];
    }
}
```



B.active controls B→C

True: reports[10,20,30], visited3
False: reports[10,20], visited2

Separate output state from future access

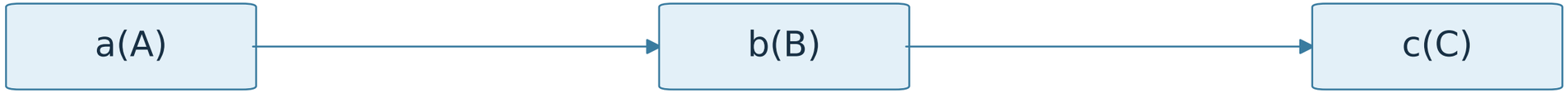
Local effect summaries feed the typed dispatch graph.

Ability	Output effect	Future-access effect
a / A	visited+=1; report10	Typed AB visit to B
b / B	visited+=1; report20	Read ordinary Boolean B.active; conditionally visit typed C
c / C	visited+=1; report30	No further visit

Here visited is total output state. It never controls a visit or cancellation in this fixture.

Compose at compile time

Implemented finite kernel: exact typed dispatch, supported guard and explicit boundary.

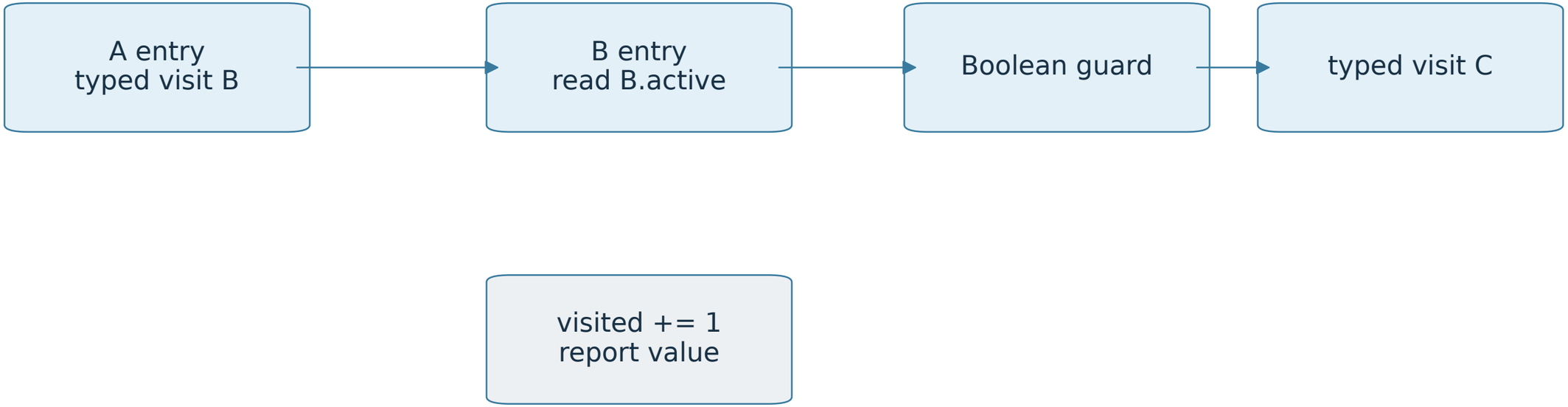


A→B ; B.active ⇒ B→C

Artifact	Recorded content
Local summary	source_type, visit, guard/guard_fields, dispatch, output effects
Region certificate	checkpoint; dispatch_edges; external_dependencies; compiled SQL template
Native method evidence	entry:0; dispatch edges1→3 and6→9 in accepted RegionGuard fixture
External bindings	Origin identity, exact runtime dispatch and actor/ACL context

A dependency graph across ability boundaries

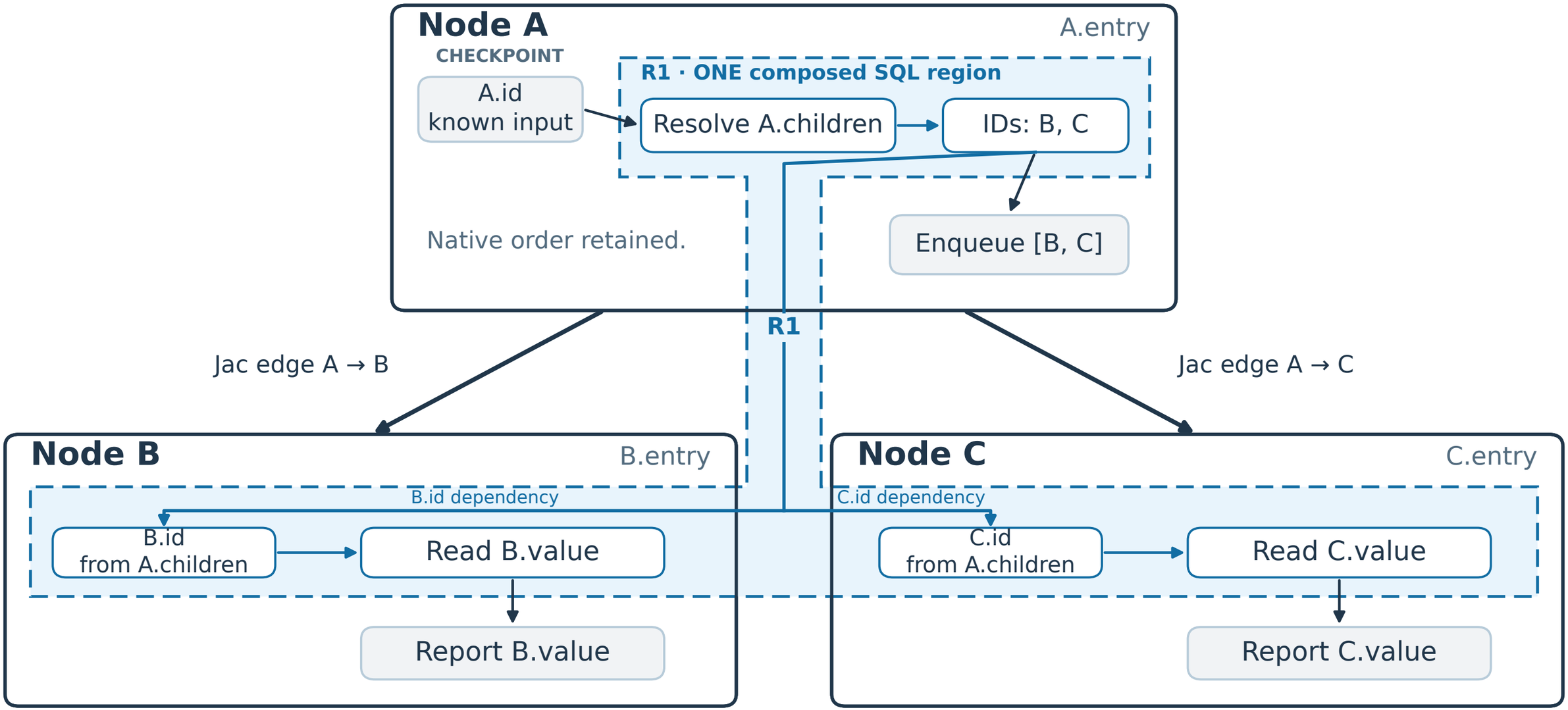
Solid access dependencies compose; output-only state stays outside access selection.



Origin and actor are ready at A entry. B.active is a node-local SQL input: it need not be known in Python before querying B. No output counter feeds the guard.

One SQL compilation region across Jac abilities

Jac Graph: A → B and A → C. The connected blue region crosses the ability boundaries.



One blue region: A path discovery + B/C payload reads

Native: enqueue, ability execution and ordered reports

No B→C future-access dependency in this example. Conceptual fanout composition; current typed region kernel is single-successor.

Move the supported guard inside SQL

Conceptual lowering of the finite example; actual SQL also validates types, ACL and arity.

```
WITH b AS (
  SELECT target_B FROM typed_AB(origin)
), c AS (
  SELECT target_C FROM b
  JOIN typed_BC(b.id) ON b.active = TRUE
)
SELECT path_pair FROM b
UNION ALL SELECT path_pair FROM c;
```

SQL boundary includes:
A→B lookup
B.active Boolean test
conditional B→C lookup

Malformed Boolean, fanout, cycle or incomplete result ⇒ decline static publication and retain native execution.
A false guard publishes no invented B→C path.

The SQL validates the second visit

Compiler proof and actual graph validation have different jobs.

Compiler supplies	SQL/runtime validates
A typed AB→B dispatch and B typed BC→C dispatch	Actual identities, types/modules and immediate arity≤1
B.active is a supported SQL-internal Boolean guard	JSON kind is Boolean; malformed values reject the static result
No intervening unsupported write/helper/cancellation	Actual order, cycle/history and resource-completion checks
Finite or homogeneous region artifact	Complete validated rows before atomic native-cache publication

Finite A→B→C uses unrolled CTE stages. Homogeneous Item→Item uses recursive lowering.

Existing checkpoint → query → ordered native cache

Runtime selects/binds an emitted artifact; native ability dispatch stays in charge.

At entry, bind origin/actor and check exact dispatch against the compiled certificate.

Execute the selected SQL, validate the complete result, then publish typed native path pairs.

Guard true: cache A→B and B→C; B entry reuses its published path.

Guard false: cache A→B only; native B runs and does not visit C.

Opaque helpers, writes, cancellation and unsupported guards form conservative boundaries.

2 · Existing replaceable admission model

These mechanism comparisons fix Always/process admission.

$$p \times \min(\text{payload}, \text{opportunity}) - \text{resolve}$$
$$- (1-p) \times \text{payload} \times \text{unused_weight} - \text{margin}$$

p=(demanded+1)/(opportunities+2); feature estimate → decision → observation interface.

Payload is dispatch-to-ready wall; opportunity is launch-to-demand, distinct from observed SQL overlap.

Fused feedback is training_observed=false. Branch frequency is collected rather than used directly in this score.

Future admission must include cold process readiness when launching a new reader. Prepared-plan reuse remains deferred.

3 · Benchmark set: 30 apps · 1–15

Source-audit inventory, one registered reader per app; current timing subset is separate.

Application	Registered reader	Future-access sibling result
approval_desk	ExpenseBoard	NO under effect summaries
asset_care	AssetBoard	NO under effect summaries
budget_ledger	BudgetBoard	NO under effect summaries
build_release	ReleaseBoard	NO under effect summaries
clinic_queue	ClinicSchedule	NO under effect summaries
course_path	LearningDashboard	NO under effect summaries
delivery_route	RouteBoard	NO under effect summaries
event_seats	EventCatalog	NO under effect summaries
garden_plan	GardenBoard	NO under effect summaries
grant_review	ReviewBoard	NO under effect summaries
help_queue	DeskBoard	NO under effect summaries
hiring_pipeline	HiringBoard	NO under effect summaries
incident_room	IncidentBoard	NO under effect summaries
issue_desk	ProjectBoard	NO under effect summaries
jacord	load_channel	NO under effect summaries

3 · Benchmark set: 30 apps · 16–30

Source-audit inventory, one registered reader per app; current timing subset is separate.

Application	Registered reader	Future-access sibling result
jdrive	VisibleFolderTree	NO under effect summaries
knowledge_map	KnowledgeBoard	NO under effect summaries
lab_samples	LabBoard	NO under effect summaries
library_loans	LibraryCatalog	NO under effect summaries
linked_list	Traverse	MAY: total-count cancellation
littlex	load_feed	NO under effect summaries
poll_assembly	AssemblyBoard	NO under effect summaries
recipe_pantry	PantryBoard	NO under effect summaries
return_counter	ReturnBoard	NO under effect summaries
sales_pipeline	SalesBoard	NO under effect summaries
stock_room	Inventory	NO under effect summaries
tournament	BracketBoard	NO under effect summaries
trip_board	Itinerary	NO under effect summaries
utility_meter	MeterBoard	NO under effect summaries
volunteer_shift	VolunteerBoard	NO under effect summaries

The additional 61 paper-operation identities

Existing selected inventory:22 TPC-H +29 SNB +5 TPC-C +5 Wikipedia.

Family	Count	Exact selected operation IDs
TPCH	22	Q1, Q10, Q11, Q12, Q13, Q14, Q15, Q16, Q17, Q18, Q19, Q2, Q20, Q21, Q22, Q3, Q4, Q5, Q6, Q7, Q8, Q9
SNB	29	IC1, IC10, IC11, IC12, IC13, IC14, IC2, IC3, IC4, IC5, IC6, IC7, IC8, IC9, IS1, IS2, IS3, IS4, IS5, IS6, IS7, U1, U2, U3, U4, U5, U6, U7, U8
TPCC	5	Delivery, NewOrder, OrderStatus, Payment, StockLevel
WIKIPEDIA	5	AddWatchList, GetPageAnonymous, GetPageAuthenticated, RemoveWatchList, UpdatePage

These 91 source-audit identities are not91 measured performance cells. This round tests four fixtures /12 n3 cells.

Finite LinkedList: a disclosed source variant

Original cutoff-bearing benchmark remains preserved separately.

```
can visit_item with Item entry {
    self.visited += 1;
    report here.value;
    visit [->:Next:->[?:Item]];
}
```

Two changes only:
1. Remove Traverse cutoff.
2. Type the target as Item.

Fixture proof: all Item nodes, at most one successor, correct incoming counts, full ordered reachability and acyclicity. visited remains a TOTAL, not a depth.

Fixed primary scope and analyzed-statistics regime

Seed42 · process1 · limit2048 · steps32768 · fresh equivalent DB/server per workflow.

Fixture	Actual measured actions
LinkedList32 /1,000	One full native Traverse per trial; Sum/Find source remains unchanged and is not invoked
Typed A→B→C	One native RegionChain, B.active=true; false tested separately in all three modes
Original LittleX	Unchanged nine primary RPCs; setup and six excluded steady views retain their own scopes

ANALYZE anchors + graph_types once after setup/views, before reset/first primary clock. WAL32/64MB. Twelve primary +3 false correctness cases;12 excluded warmups +36 measured trials.

Actual DB protocol time

Median[min,max]ms, n=3. Every primary summed interval equals its interval union.

Fixture	Off	9282	Interability
LinkedList32	36.751 [25.926, 39.299]	6.509 [6.502, 7.355]	7.701 [7.355, 7.713]
LinkedList1,000	794.247 [773.849, 795.006]	31.441 [30.736, 31.838]	59.877 [58.408, 64.344]
A→B→C	3.788 [3.506, 4.019]	7.228 [7.190, 7.264]	7.620 [7.571, 8.757]
LittleX	41.946 [41.466, 47.872]	53.754 [53.484, 55.187]	56.560 [53.297, 57.187]

Actual primary HTTP response wall

Median[min,max]ms, n=3; client start through full response-body completion.

Fixture	Off	9282	Interability
LinkedList32	60.516 [43.848, 64.132]	652.823 [652.582, 654.526]	254.941 [251.715, 259.642]
LinkedList1,000	1,240.857 [1,210.998, 1,254.452]	30,841.349 [29,852.764, 30,987.718]	726.128 [720.944, 733.943]
A→B→C	8.449 [8.323, 8.966]	444.501 [442.302, 453.240]	241.448 [239.783, 246.888]
LittleX	98.207 [94.999, 106.658]	1,586.361 [1,572.528, 1,588.528]	1,592.227 [1,576.288, 1,595.083]

LL/AB each have one request. LittleX sums nine response intervals. Explicit drain and client gaps remain separate; these are instrumented runs.

Every SQL call is counted

Actual data + controls = TOTAL, identical across all three measured repeats.

Fixture	Off	9282	Interability
LinkedList32	68 + 7 = 75	7 + 8 = 15	7 + 8 = 15
LinkedList1,000	2004 + 7 = 2011	7 + 8 = 15	7 + 8 = 15
A→B→C	9 + 7 = 16	9 + 8 = 17	7 + 8 = 15
LittleX	90 + 63 = 153	82 + 69 = 151	78 + 69 = 147

Includes all foreground/background path, payload, identity/catalog and transaction-completion SQL.

LL1,000: less interpreted continuation work

Recorded abstract work counters; all actual native visits remain1,000.

Recorded quantity	9282	Interability
Continuation records	2,000	0
Abstract executions	999,000	0
Continuation path records	1,000,000	0
Native plans	1,000	1
Submitted / promoted payloads	999 /999	999 /999

Static region selection bypasses the interpreter. These counters measure work quantity, not CPU time.

Process readiness is a measured fixed wall cost

One native constructor clock covers imports/configuration through the ready handshake.

Primary case	Unique child PIDs/trial	9282 startup sum ms	New startup sum ms
LinkedList32	1	190.879 [188.795, 192.305]	192.903 [189.638, 196.703]
LinkedList1,000	1	190.760 [186.224, 196.071]	191.261 [190.628, 193.422]
A→B→C	1	189.747 [188.285, 192.037]	190.615 [189.063, 191.696]
LittleX	6	1,148.471 [1,138.714, 1,148.924]	1,144.448 [1,139.219, 1,145.275]

~190ms per PID; LittleX six PIDs total~1.14s versus~1.59s response wall. Timers exclude later lazy DB connect/load. No aligned critical-path percentage or CPU subtraction.

Natural scope cleanup and process reuse

Source order explains the lifecycle; individual IPC/close durations were not retained.

configured_dependency_prefetch wraps native spawn; finally joins its worker threads.

The enclosing dependency scope closes ProcessReaders before spawn returns; Session.close also cleans a remaining pool.

Recorded parent IPC workers: LL1 each; A→B→C2→1; LittleX6 each. Actual child SQL/backend identities are verified.

All 48 trials show zero overlapping SQL protocol intervals on distinct backends.

Proposed persistent readers need per-request transaction, actor, tracing and context reset.

LittleX: Stage2 templates, a separate mechanism

Static interability region queries=0 in this workload.

Measured quantity	9282	Interability
TOTAL SQL	151	147
Data /control SQL	82 /69	78 /69
HTTP response-sum median ms	1586.361	1592.227
DB protocol median ms	53.754	56.560

Pure visit-prefix sites receive later frontier attempts using existing continuation/barrier logic. Stage2 is distinct from the new typed single-successor kernel.

Matched recursive SQL diagnostic

Separate matched native diagnostic; primary n3 results stay unchanged.

Mode	Planning first/repeat ms	Execution first/repeat ms	Shared hits	Temp writes (blocks)
baseline9282	0.773 / 0.658	14.346 / 14.107	16136	0
interability	1.111 / 0.897	41.481 / 40.506	22136	985

Matched native recapture; SERIALIZABLE, explicit pre-workflow ANALYZE, same-connection first/repeat EXPLAIN. Census warms buffers; separate from primary n=3.

Why this recursive SQL does more work

Observed plan differences; no per-feature timing ablation was performed.

Quantity / structure	9282	Interability
Recursive state	origin, depth	origin, phase, depth, growing seen UUID array
Extra source-node PK validation	None	2 × 1,000 loops; 6,000 extra shared hits
Root shared hits / reads	16,136 / 0	22,136 / 0
CTE temporary written blocks	0	985 at work_mem 4 MB
First / repeat execution ms	14.346 / 14.107	41.481 / 40.506

Growing cycle history and extra validation are concrete optimization targets. Preserve cycle/completeness/ACL correctness; inclusive buffer counters are not additive.

Next improvements and larger-stage follow-up

Proposals are separate from the tested runtime and fixed-admission results.

Use the matched-plan evidence to target recursive SQL costs while preserving certificate checks.

Explore process reuse with explicit transaction/actor/context/tracing reset and cold-start-aware admission.

Extend helper/effect summaries and dominated external guard bindings beyond the first typed kernel.

Keep prepared-statement reuse and adaptive region admission as separate evaluated changes.

Larger LittleX 50×20 / LL 2048 follow-up is paused at user request; no larger runs were started.

Reproducible measured scope

All primary inputs, traces, semantic proofs, source versions and analysis remain bound.

Runtime:9282 baseline versus clean addd3a4; Off uses the new runtime. Common process1/budgets and analyzed-statistics regime.

15 native correctness cases, then48 fresh trials:12 excluded warmups and36 measured samples.

Actual SQL sum/union/counts independently reaggregated; primary instrumented HTTP clocks audited.

Compact native counters retain abstract work and startup observations without quadratic trace serialization.

Editable renderer, figure inputs, source/analysis manifests and completed/excluded inventory accompany the PDF.

4 · Sibling dependence of future access

Shared output does not automatically imply interference between sibling visits.

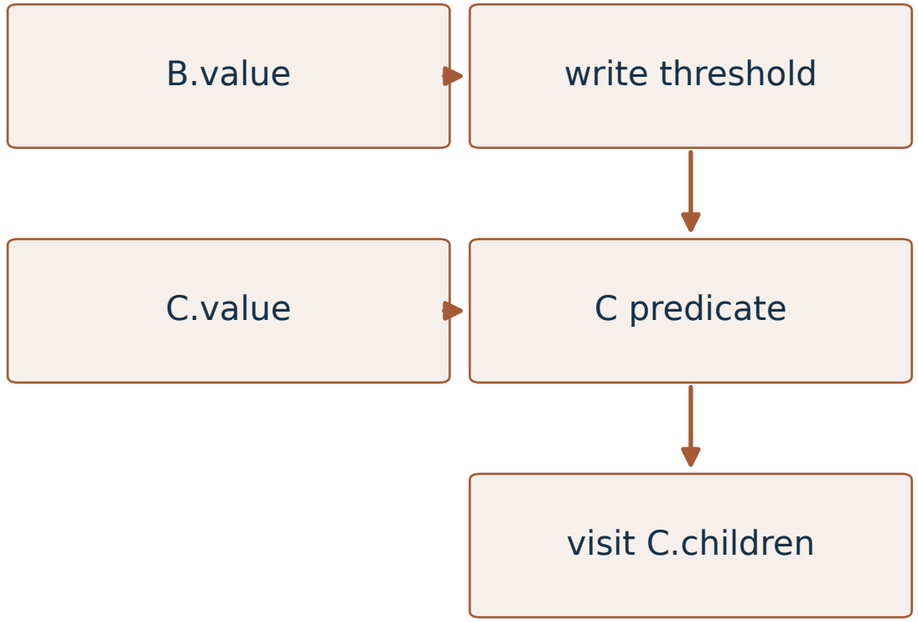
Witness	Static interpretation
self.total += here.value; visit fixed Next	Output-only reduction if total is never read to choose access
if self.visited>=limit { disengage; }	MAY dependence: an earlier sibling can cancel later queued work; count is total, not depth
seen guard before visit /graph read	MAY dependence: earlier visits can suppress a later access
Opaque external helper /storage side effect	Requires an effect contract; unknown is not a theorem of impossibility

A sibling write can change the later access

Conceptual A→{B,C} example, with B scheduled before C.

```
B.entry:
    self.threshold = B.value

C.entry:
    if C.value > self.threshold:
        visit C.children
```



The write in B is a dependency of C’s future visit. Keep this boundary until the required state transition and ordering can be represented soundly.

What the 91-source audit establishes

Closed-world ordinary-field/effect reasoning; not an implemented universal analyzer.

Registered readers:29 NO under explicit summaries; original cutoff-bearing LinkedList exposes MAY cancellation.

Additional helpers/business walkers have their own seen guards, writes, queue and phase boundaries.

TPC-H whole-operation view:16 NO,5 phase MAY, and Q20 conditional materialization dependence.

Within a fixed TPC-H sibling stage, immutable join maps differ from feedback; phase/reduction dependencies can be relationally lowerable.

Current unsupported syntax is separate from semantic dependence and from whether a static summary can prove it.

Start with sibling-independent future access

First analyze and compose regions with proved sibling-independent future access.

Analysis outcome	Initial treatment
Proved NO sibling feedback	Compose supported regions; keep original native queue/output semantics.
MAY: shared state reaches a visit, guard or cancellation	Retain an explicit boundary. Later SQL lowering needs the correct state and ordering semantics.
Unknown helper, descriptor or side effect	Require a bounded effect summary or fall back; unknown does not mean impossible.
Output-only reduction	Allow it only after proving it cannot feed future accesses. Total visited is not recursion depth.

The small chain and guarded A→B→C validate the first typed kernel. The wider source audit guides extensions; it is not evidence of implementation coverage for all 91 cases.